RoboFlow - Resumo para Apresentacao

Documento de apoio para a apresentacao do projeto de Compiladores

1. Resposta direta sobre o estado do codigo

As analises lexica e sintatica estao implementadas e funcionam para os exemplos atuais do projeto. Os comandos abaixo foram executados com sucesso:

```
python examples\demo_compilador.py examples\rota_basica.rf
python examples\demo_compilador.py examples\rota_completa.rf
```

Os dois exemplos geraram uma Arvore de Sintaxe Abstrata (AST) valida. O exemplo completo tambem processou corretamente um condicional `if/else`.

O projeto ainda nao e um compilador completo. O estado atual e:

```
Codigo .rf -> Lexer -> Tokens -> Parser -> AST -> Analise semantica -> Geracao de codigo -> ESP32
                pronto     pronto     pronto       pendente             pendente
```

2. Proposta do projeto

O RoboFlow e uma Linguagem de Dominio Especifico (DSL) para descrever o comportamento de um AGV, isto e, um veiculo industrial que percorre uma linha e executa acoes em estacoes de trabalho.

O projeto separa duas responsabilidades:

- Controle continuo: o ESP32 le sensores, segue a linha com um controlador PID, controla motores e aplica o veto de seguranca.
- Decisao discreta: o programa RoboFlow define o que o AGV deve fazer ao chegar a cada estacao: parar, esperar, virar, sinalizar ou retomar o seguimento da linha.

Exemplo:

```
station "carga" {
    on_arrival {
        stop()
        wait_signal("carga_completa", timeout: 30s)
        resume_line_following()
    }
}
```

Em outras palavras: ao chegar a estacao `carga`, o AGV para, espera pelo sinal de carga completa por no maximo 30 segundos e volta a seguir a linha.

3. A linguagem RoboFlow

Os principais comandos da DSL sao:

Comando	Significado
`stop()`	Para o AGV.
`set_speed(valor)`	Define a velocidade de seguimento.
`turn_left()` / `turn_right()`	Solicita uma manobra.
`continue_straight()`	Continua em frente.
`wait_signal(nome, timeout: t)`	Aguarda um evento externo.
`signal_buzzer()` / `signal_light()`	Sinaliza para operadores.
`log(mensagem)`	Registra uma mensagem.
`resume_line_following()`	Devolve o controle ao PID de seguimento de linha.

Tambem existem condicionais:

```
if next_destination == "linha_2" {
    turn_right()
} else {
    continue_straight()
}
```

E existe uma estacao `default`, usada como comportamento de seguranca para identificadores de estacao desconhecidos.

4. Analise lexica

A analise lexica e a primeira fase do compilador. Ela recebe caracteres do arquivo `.rf` e os agrupa em unidades chamadas tokens.

Para esta entrada:

```
wait_signal("ok", timeout: 30s)
```

O lexer produz, conceitualmente:

```
KEYWORD(wait_signal) LPAREN STRING(ok) COMMA KEYWORD(timeout)
COLON DURATION(30.0) RPAREN
```

O lexer reconhece palavras reservadas, identificadores, strings, numeros, duracoes, operadores, chaves, parenteses, virgulas e dois-pontos. Espacos e comentarios iniciados por `//` sao descartados.

Regras importantes

- Palavra reservada x identificador: palavras como `station`, `stop`, `if` e `else` sao classificadas como palavras reservadas. Nomes como `next_destination` sao identificadores.
- Casamento mais longo (maximal munch): `30s` forma um unico token de duracao, e nao `NUMBER(30)` seguido de `ID(s)`.
- Diagnosticos: o lexer informa linha e coluna quando encontra um caractere invalido ou uma string nao terminada.

Exemplo de erro lexico:

```
stop(@)
```

Resultado esperado:

```
[Erro lexico] linha 1, coluna 33: caractere inesperado '@'
```

No codigo, a especificacao dos tokens esta em `compiler/roboflow/tokens.py` e o scanner esta em `compiler/roboflow/lexer.py`. A documentacao define os tokens por expressoes regulares; a implementacao usa um scanner manual caractere a caractere equivalente a essa especificacao.

5. Analise sintatica

Depois da tokenizacao, o parser verifica se os tokens aparecem em uma ordem permitida pela gramatica da linguagem.

Uma versao simplificada da gramatica e:

```
program      ::= station_decl*
station_decl ::= "station" (STRING | "default") "{" "on_arrival" "{" stmt_list "}" "}"
stmt_list    ::= stmt*
stmt         ::= command_call | if_stmt
command_call ::= KEYWORD "(" arg_list? ")"
if_stmt      ::= "if" condition "{" stmt_list "}" ("else" "{" stmt_list "}")?
condition    ::= ID "==" (STRING | ID)
```

O parser foi implementado com descida recursiva. Isso significa que ha uma funcao para cada producao importante da gramatica, como `station_decl`, `command_call`, `if_stmt` e `condition`.

Exemplo de erro sintatico: se faltar uma chave para fechar a estacao, o parser interrompe a analise e informa a linha, a coluna, o token proximo e o item esperado.

```
[Erro sintatico] linha 1, coluna 36, perto de 'EOF':
esperado '}' para fechar a estacao
```

O parser esta em `compiler/roboflow/parser.py`.

6. AST - Arvore de Sintaxe Abstrata

Quando a entrada esta sintaticamente correta, o parser cria uma AST. A AST guarda a estrutura logica do programa e elimina detalhes que nao sao necessarios nas fases seguintes, como parenteses e chaves.

Exemplo de estrutura para uma bifurcacao:

```
PROGRAM
└── STATION "bifurcacao_A"
    ├── IF
    │   ├── CONDITION: next_destination == "linha_2"
    │   ├── THEN: turn_right()
    │   └── ELSE: continue_straight()
    └── resume_line_following()
```

Os principais tipos de no sao `ProgramNode`, `StationNode`, `CommandCallNode`, `IfStmtNode`, `ConditionNode`, `LiteralNode` e `IdentifierNode`. Eles estao definidos em `compiler/roboflow/ast_nodes.py`.

7. Diferenca entre erro lexico, sintatico e semantico

Tipo de erro	Exemplo	Responsavel
Lexico	`stop(@)`	Lexer: `@` nao pertence ao alfabeto da linguagem.
Sintatico	Falta uma chave `}`	Parser: os tokens nao obedecem a gramatica.
Semantico	`set_speed(999999)`	Analisador semantico: a estrutura e valida, mas o valor nao faz sentido para o AGV.

Esta distincao e essencial na apresentacao: um programa pode estar bem escrito do ponto de vista lexico e sintatico, mas ainda ser invalido para o dominio do robo.

8. O que ainda falta

A proxima fase e a analise semantica. Ela deve validar:

- existencia de exatamente uma estacao `default`;
- nomes de estacao sem repeticao;
- comandos pertencentes a biblioteca da linguagem;
- quantidade e tipos corretos dos argumentos;
- limites fisicos, como velocidade maxima;
- uso de variaveis de contexto em condicionais;
- termino seguro de cada caminho de execucao, com `resume_line_following()` ou uma espera segura.

Depois disso, o compilador devera criar uma representacao intermediaria e gerar codigo para o alvo escolhido. A documentacao do repositorio ainda possui uma divergencia: alguns textos mencionam codigo Python no Raspberry Pi e o roadmap mais recente prioriza C++ para o ESP32. Para a apresentacao, a formulacao segura e: o repositorio atual implementa o front-end RoboFlow; a geracao de codigo e a integracao embarcada sao proximas etapas.

9. Observacoes tecnicas atuais

- Os dois programas de exemplo foram aceitos pelo lexer e pelo parser.
- O exemplo `rota_completa.rf` inclui `if/else` e foi convertido para AST corretamente.
- A demonstracao atual imprime enderecos de memoria na parte chamada de "tabela de tokens", porque a classe `Token` ainda nao possui uma representacao textual (`__str__` ou `__repr__`). O lexer funciona, mas essa visualizacao deve ser melhorada antes da apresentacao.
- Ainda nao existe uma suite automatizada de testes no repositorio.
- O lexer aceita `30.` como numero, embora a especificacao documentada exija digitos apos o ponto decimal. Este e um ajuste pequeno recomendado.

10. Roteiro curto para apresentacao

1. Apresente o problema: um AGV precisa seguir uma linha e tomar decisoes em estacoes. 2. Explique a separacao: PID e seguranca ficam no ESP32; a DSL descreve o comportamento de estacao. 3. Mostre um programa RoboFlow curto. 4. Explique o fluxo: arquivo `.rf` -> tokens -> AST -> futuras fases. 5. Mostre a analise lexica com `30s`, palavras reservadas e um erro de caractere invalido. 6. Mostre a analise sintatica com a gramatica e uma AST. 7. Execute `python examples/demo_compilador.py examples/rota_completa.rf`. 8. Finalize com o estado real: lexer, parser e AST prontos; semantica, geracao e integracao ainda pendentes.

Frase de encerramento sugerida:

> O lexer verifica se os elementos da linguagem existem, o parser verifica se eles formam uma estrutura valida e a analise semantica verificara se essa estrutura e segura e faz sentido para o AGV.